ASWP: Actual Training & Benchmark Execution Plan

A compute-conscious, reproducible plan for testing Annealed Stochastic Weight Perturbation against ordinary training

Goal: determine whether ASWP improves a fixed Transformer training setup, using free/common data and a controlled comparison, while protecting an approximately $30/month Modal credit budget.

Prepared: October 17, 2026

Project: kragent66-glitch/aswp

Important status note

The repository's analysis work has improved, but the actual training harness and ASWP module are not yet implemented. This document is the execution blueprint; it does not claim a model has been trained or that ASWP works.

Recommended path: TinyStories → small decoder-only Transformer → baseline/ASWP matched pilot → measured cost → preregistered confirmation only if affordable.

1. Executive Summary

The scientific question is not whether a model trained with ASWP can produce plausible text. It is whether adding the ASWP perturbation method improves validation performance compared with the **same model trained normally**, under matched conditions.

The most compute-efficient fair comparison is a paired experiment: run the same architecture, data order, tokenizer, optimizer, training budget, and evaluation pipeline with ASWP disabled versus enabled. This does require training a baseline. Comparing only against an unrelated public checkpoint is cheaper, but it cannot isolate the effect of ASWP because model size, data, training duration, tokenizer, and recipe would differ.

Decision	Recommendation	Reason
Dataset	TinyStories, fixed revision and fixed train/validation split	Small, widely used for tiny language-model experiments; quick iteration.
Tokenizer	GPT-2 BPE as a candidate; freeze exact tokenizer files and hashes before runs	Common reference point. Do not mix tokenizers between arms.
Model	Small decoder-only Transformer, initially ~1–3M parameters for plumbing; then a predeclared ~10M screening model if affordable	Small models validate correctness cheaply; the screen tests the actual intervention at a more meaningful scale.
Comparator	Same model trained normally, paired seed by paired seed	Only clean way to estimate the effect of ASWP itself.
Compute	Use a short GPU benchmark first; do not assume $30 covers eight paired runs	Runtime, GPU price, memory, storage and retries determine actual cost.
Primary metric	Final deterministic validation negative log-likelihood (NLL)	Comparable, reproducible metric; lower is better.

Go/no-go: do not begin confirmatory runs until the training harness passes Phase 0, all required protocol fields are locked, and a measured cost estimate fits the available credits with a reserve.

2. What ASWP Changes—and What It Does Not

ASWP is a training-time weight-perturbation method for a Transformer, not a completely different model family. The comparison should therefore keep the Transformer architecture fixed and change only the predeclared ASWP intervention. The exact implementation in the repository's research protocol must be followed; do not improvise a new noise rule mid-experiment.

- **Traditional baseline:** ordinary forward/backward passes and optimizer updates; no ASWP perturbation.
- **ASWP treatment:** same forward/backward and optimizer setup, plus the protocol-defined Gaussian, RMS-relative perturbation with a cosine schedule.
- **Critical control:** the perturbation schedule must reach exactly zero at the final step, and the perturbation must be applied at the exact point and frequency specified by the protocol.
- **Important distinction:** ASWP does not remove the need for a baseline. It is the treatment whose incremental effect you are trying to estimate.

3. Dataset Recommendation

Primary recommendation: **TinyStories**, with a fixed dataset revision and a deterministic preprocessing pipeline. It is suitable for low-cost debugging and small-model language-model experiments. Its simplicity is a limitation: a positive result would establish evidence on this task and scale, not prove that ASWP helps all language models or broad real-world text.

Dataset setup to freeze

- Choose one canonical public TinyStories dataset source and record the exact repository/dataset identifier, revision or commit, license information, and download URL.
- Use a fixed training split and a fixed validation split. Never use validation text for training or model selection beyond the explicitly predeclared exploratory screen.
- Record the raw file hashes and a manifest of filenames, byte sizes, and row counts.
- Define text normalization, handling of empty or malformed records, document boundaries, end-of-text tokens, packing strategy, sequence length, and whether sequences may cross document boundaries.
- After tokenization and packing, hash the final validation token stream. The validation stream must be byte-for-byte identical for every baseline and ASWP run.
- Keep a held-out test split untouched until the final reporting stage if the selected dataset provides one; otherwise state clearly that validation is the primary endpoint.

Why not start with FineWeb-Edu?

FineWeb-Edu is a reasonable later transfer/generalization dataset, but it adds data volume, preprocessing complexity, and compute cost. Do not make it the first gate while the training code is unverified. A TinyStories result should be described as a small-scale controlled test, not as proof of general improvement.

Licensing and reproducibility

Before downloading or publishing a derived dataset artifact, verify the current dataset card and license. Store the source citation and the exact revision in the experiment manifest. Dataset availability and license terms can change; do not rely on a dataset name alone as a reproducible specification.

4. Model, Tokenizer, and Training Configuration

Use a small, conventional decoder-only Transformer as the fixed host architecture. ASWP is the intervention; changing the model architecture at the same time would confound the result.

Item	Pilot choice / rule to lock	Must be identical across arms?
Architecture	Decoder-only Transformer; write down layer count, hidden size, attention heads, MLP width, positional encoding, normalization, bias policy and tied/untied embeddings.	Yes
Tokenizer	Candidate: GPT-2 BPE. Pin tokenizer repository, revision, vocab/merges or tokenizer JSON hashes, special tokens and library version.	Yes
Context length	Choose one value based on memory and task; freeze it before comparison.	Yes
Initialization	Same initialization recipe and paired initialization seed for each baseline/treatment pair.	Yes
Optimizer	Choose one optimizer and freeze all settings: learning rate, betas, epsilon, weight decay, clipping, schedule, warmup and batch size.	Yes
Training budget	Fix optimizer steps and total training tokens; do not stop one arm early because its loss looks better.	Yes
Precision	Choose a supported precision and record it. Construct noise in FP32 as specified by the protocol; measure realized perturbation after casts.	Yes, except treatment-only noise operations
Evaluation	Deterministic evaluation mode, fixed validation token stream, no dropout, same checkpoint step.	Yes
ASWP settings	Gaussian distribution, RMS-relative scale, cosine schedule, target module list, sigma0, update frequency and final-step behavior.	Treatment settings fixed once selected

Parameter-count stages

- **Stage A: 1–3M parameters.** Use only for code correctness, shape checks, gradients, deterministic replay, and smoke tests. Do not present this stage as decisive scientific evidence.
- **Stage B: around 10M parameters.** Use as the initial screening scale only if the measured throughput and budget allow it. Keep the model configuration fixed across baseline and ASWP candidates.
- **Stage C: confirmatory model.** Use the same architecture and budget as the selected screening configuration. Do not increase model size or token budget after seeing a favorable result without declaring a new exploratory experiment.

Important model-selection rule

Do not tune baseline and treatment independently. If a configuration needs tuning, use the same screening budget and decision rule for each candidate. The final confirmatory setting must be selected using the written rule, then frozen before confirmation.

5. Experiment Design: The Minimum Fair Comparison

Use paired seeds. Each pair has a baseline run and a treatment run with the same paired seed and the same deterministic data-order and initialization setup. Their only intended difference is whether the ASWP perturbation is enabled. The repository currently defines eight paired seeds in `config/manifest.json`; preserve the committed values unless a documented protocol revision is made.

Control	Baseline	ASWP
Model architecture	Identical	Identical
Dataset and token stream	Identical	Identical

Control	Baseline	ASWP
Tokenizer and context length	Identical	Identical
Initialization / data-order seeds	Paired seed rules	Same paired seed rules
Optimizer and LR schedule	Identical	Identical
Steps and tokens seen	Fixed budget	Same fixed budget
Noise perturbation	Disabled	Protocol-defined perturbation
Validation procedure	Deterministic, same step	Deterministic, same step

Why an external public checkpoint is not enough

A public pretrained checkpoint may be a useful contextual reference, but it is not an apples-to-apples control unless its architecture, data, tokenizer, training budget and evaluation conditions match. It cannot tell you whether ASWP caused a difference. If you use one, label it as an external reference, not as the primary baseline.

6. Metrics and Success Criteria

- **Primary endpoint:** deterministic validation NLL at one fixed, preregistered optimizer step. Lower is better.
- **Paired difference:** d_i = baseline validation NLL – ASWP validation NLL. Positive values favor ASWP.
- **Report all pairs:** all eight paired differences, means, medians, SDs, and individual-pair plot; never show only an aggregate bar.
- **Exact test:** use the repository's exact paired sign-flip/randomization test and its locked one-sided alpha = 0.05 convention.
- **Practical threshold:** the SAP defines $\delta = 1.0 \times SD_{baseline}$; report this threshold and its SD_delta equivalent. Do not conflate SD_baseline and SD_delta.
- **Locked favorable decision:** point estimate exceeds δ and the one-sided 95% randomization lower bound exceeds zero, with no unacceptable stability or cost regression.
- **Secondary metrics:** training loss curve, throughput, peak GPU memory, instability count, skipped/failed steps, total GPU seconds, wall-clock time and cost.
- **Exploratory only:** best checkpoint, minimum-over-time NLL, alternative thresholds, extra metrics or post-hoc subsets must not replace the locked primary endpoint.

A positive result is not guaranteed

The analysis plan itself indicates that the design may have limited power at the practical δ threshold under the worst predeclared variance-ratio conditions. Eight pairs are a target, not a guarantee of decisive evidence. An inconclusive result is valid and should be reported as inconclusive, rather than changing the threshold after the runs.

7. Training Harness: What Must Be Built

The repository's analysis scripts are not a training system. Before any experiment, implement the actual model, data loader, optimizer loop, ASWP module and run ledger.

Component	Required behavior
Configuration	One validated config file per run; all hyperparameters explicit; config serialized into each run directory.
Dataset loader	Download/cache, verify revision and hashes, deterministic split, tokenizer and packed-token stream, no silent fallback.
Model	Small decoder-only Transformer with an explicit architecture description and parameter count.
ASWP module	Apply protocol-defined perturbation to the eligible matrices; support sigma0, cosine schedule, FP32 noise construction, final-step sigma exactly zero.
RNG management	Independent role streams: global, init, data-loader, dropout and perturbation. Perturbation must use a dedicated `torch.Generator` as specified.
Training loop	Fixed optimizer-step/token budget; gradient accumulation if needed; clipping; schedule; consistent optimizer update order.
Evaluation	Deterministic evaluation; model in eval mode; no gradients; fixed validation stream; report token-weighted NLL.
Checkpointing	Save model, optimizer, scheduler, global step, RNG states, config, code commit and data manifest; test resume equivalence.
Run ledger	Record run ID, seed, arm, sigma, commit, hashes, start/end, tokens, steps, losses, throughput, peak memory, instability and cost.
Artifacts	Keep raw metrics and logs; write a machine-readable JSON/CSV summary; do not overwrite prior runs.

Suggested run directory

`runs////` with `config.json`, `manifest.json`, `metrics.jsonl`, `train.log`, `checkpoint/`, and `summary.json`. Store immutable run outputs; use a separate analysis script to aggregate paired results.

8. Mandatory Phase 0 Tests (Before Scientific Runs)

1. **Zero-noise equivalence:** ASWP with sigma0 = 0 must match the ordinary baseline path within a defined numerical tolerance for logits, loss and gradients. If the implementation intentionally changes any operation order, document and quantify that difference.
2. **Schedule correctness:** test initial scale, monotonic annealing, expected intermediate values and exact zero at the final training step.
3. **Target-module correctness:** only protocol-eligible matrices are perturbed; embeddings, output head, biases, norms, tied weights and excluded modules remain untouched.
4. **Shape/device/dtype correctness:** CPU and chosen GPU, FP32 and selected mixed precision, batch size one and normal batch, short final batch, gradient accumulation.
5. **Gradient sanity:** gradients are finite; no accidental detachment; optimizer updates occur as expected; noise is applied at the intended point in the forward/update lifecycle.
6. **RNG isolation:** enabling ASWP must not silently change data order or dropout randomness for unrelated streams. Same config and seed must reproduce the run within declared deterministic limits.
7. **Checkpoint/resume:** an uninterrupted run and a split-save-resume run reach equivalent model/optimizer/RNG state and metrics within tolerance.
8. **Deterministic evaluation:** repeated validation of the same checkpoint produces identical results under the stated hardware/software policy.

9 **Stability handling:** define and test behavior for NaN/Inf loss, exploding gradients, OOM, interruption, and infrastructure retries. Do not silently drop unfavorable unstable runs.

10 **Cost measurement:** run a representative timed segment, measure steady-state tokens/second, peak memory and wall-clock overhead with ASWP off and on.

Do not use the tiny debug run as evidence of efficacy. Phase 0 is an implementation gate, not the result.

9. Staged Execution Plan

Stage	Work	Exit gate
0 — Freeze plumbing	Dataset download, tokenizer, model, deterministic packing and baseline-only smoke test.	Hashes verified; batches and labels correct; loss finite.
1 — Phase 0 correctness	All tests above, including zero-noise equivalence, RNG isolation, resume and deterministic evaluation.	All gates pass; no known correctness defect.
2 — Cost pilot	Short baseline and ASWP runs on the intended GPU; measure tokens/sec, memory and cost.	Measured budget and safe reserve established.
3 — Exploratory screen	Run predeclared sigma candidates and required control arms using equal effort/budget; record every run.	Apply locked selection rule; select sigma0 without looking at confirmatory outcomes.
4 — Freeze protocol	Complete checklist fields, hashes, optimizer settings, step/token budget, checkpoint step, selected sigma0 and parent exploratory run ID; commit and tag protocol.	Checklist has no pre-run blockers.
5 — Confirmation	Eight paired seeds, baseline and ASWP, fixed config and fixed evaluation endpoint.	All pairs accounted for; failures retained and documented.
6 — Analysis and release	Run locked analysis, publish pair-level results, config, code commit, hashes, cost and deviations.	Conclusions match preregistered decision language.

10. Compute Budget: Protect the $30 Credit

Do not assume a particular Modal GPU price or that $30 covers the full experiment. Prices, credit eligibility and GPU availability can change. Verify current pricing in your Modal account immediately before running; the measured runtime is the basis of the budget.

Budget formula

Estimated GPU cost = (GPU hourly rate) × (measured GPU-hours for all runs). Add any applicable storage, volume, bandwidth, image-build, CPU-only or other charges, and account for failed/retried runs. Use the actual billed price shown by Modal, not a remembered price.

Measure before scaling

1. Run a representative short baseline segment and record GPU time, tokens/second and peak memory.
2. Run the same segment with ASWP enabled and measure its overhead.
3. Estimate one full run using the frozen token/step budget; add evaluation, startup, checkpointing and expected retries.
4. Calculate the cost of screening all candidates and controls, then the cost of eight paired confirmation seeds (16 runs, baseline plus ASWP per seed).
5. Reserve at least 20–30% of the available credit for overhead, retries and measurement error. If the estimate exceeds the usable credit, reduce the exploratory budget or defer confirmation; do not quietly change the confirmatory design after seeing results.

Budget bucket	Purpose	Spending rule
Phase 0	Correctness and tiny smoke tests	Keep very short; CPU where practical, GPU only for device-specific checks.
Cost pilot	Measure actual GPU speed and ASWP overhead	One short matched baseline/treatment segment.
Screening	Choose sigma0 and verify controls	Use only the written candidate list and equal budgets; log all candidates.
Confirmation	Eight paired seeds	Run only if total cost plus reserve fits. Do not start if partial completion would make the study uninterpretable.
Reserve	Retries, failures, unexpected overhead	Protect 20–30% of credits if possible.

A $30 cap is a resource constraint, not a reason to weaken the evidence silently. If the full eight-pair design is unaffordable, first publish the engineering and screening result as exploratory, then obtain more compute or formally revise the study before confirmatory runs.

11. Benchmarking Plan and Interpretation

The core benchmark is a controlled within-project baseline, not a leaderboard comparison.

- Primary comparison: baseline versus ASWP with identical model, data, tokenizer, training tokens, optimizer and evaluation.
- Optional external context: report a public pretrained model only if its tokenizer, evaluation protocol and model identity are disclosed; treat it as context, not causal evidence.
- Optional generalization phase: after the TinyStories study is complete, repeat a separately designed experiment on a broader dataset such as FineWeb-Edu if compute allows.
- Report validation NLL and uncertainty, but also throughput and stability. An NLL improvement that costs substantially more compute or destabilizes training may not be practically useful.
- Avoid cherry-picking checkpoints, seeds, prompts or samples. Keep the final endpoint fixed and include all paired results.

12. Preregistration Checklist

The repository's `docs/06-preregistration-checklist.md` is the authoritative operational checklist. Before confirmation, ensure every pre-run item is committed and the checklist explicitly says the study is ready.

- Dataset identifier, revision, license reference, raw data hashes, train/validation split and final validation token-stream hash.
- Tokenizer identity, version, vocabulary/special-token files and hashes.
- Model source commit, architecture, parameter count and eligible module list.
- Optimizer, learning-rate schedule, warmup, weight decay, clipping, dropout, batch size, sequence length, precision, optimizer steps and total tokens.
- Fixed endpoint/checkpoint step and deterministic validation procedure.
- Selected sigma0, selection rule and exploratory parent run ID.
- Frozen paired seeds and per-role RNG mapping; resolve and document the manifest derivation wording consistently with issue #10.
- Analysis version, primary metric, exact test, alpha, practical threshold, instability rule, cost budget and protocol deviation policy.
- Commit hash, signed/dated protocol record, and a release tag before confirmation.

13. Reporting Template

For every run, save the following fields so results can be independently checked:

```
experiment_id, run_id, arm, paired_seed, per-role seeds, git_commit, dataset_revision, data_hash,
tokenizer_hash, config_hash, model_architecture, parameter_count, sigma0, step, tokens_seen, train_loss,
validation_nll, throughput_tokens_per_second, peak_gpu_memory, instability_events, start_time, end_time,
gpu_type, billed_gpu_seconds, estimated_cost, checkpoint_path, protocol_deviations.
```

Final report should include: all eight paired d_i values; baseline and ASWP validation NLL; mean and median paired improvement; SD_baseline and SD_delta; exact sign-flip p-value; one-sided 95% randomization lower bound; δ in both units; instability and throughput comparisons; GPU cost; code/data/config hashes; and every protocol deviation.

14. Current Repository-Specific To-Do List

Priority	Task	Why it matters
P0	Review and merge PR #17 only after checking final diff and successful Actions run.	Keeps analysis fixes, seed manifest and checklist in main.
P0	Update PR description's stale test count (PR text says 37; checklist says 67) after verifying actual CI output.	Documentation should reflect what CI really ran.
P0	Resolve seed-list protocol wording: issue #10 says seeded RNG; implementation derives seeds deterministically from SHA-256.	Avoid an unrecorded mismatch between the requirement and implementation.
P0	Implement the model, data pipeline, ASWP module and training loop.	No real training can happen without this.
P0	Complete Phase 0 gates and record measured runtime/cost.	Prevents spending the credit on a broken or unaffordable experiment.
P1	Fill all data/model/training configuration fields and hashes.	Makes the study reproducible.
P1	Run exploratory screen and freeze sigma0/parent run ID.	The treatment configuration must be selected before confirmation.
P1	Commit/tag final preregistration; ensure checklist has no pre-run blockers.	Prevents post-hoc changes.
P1	Run eight paired confirmation seeds if affordable; otherwise stop and formally revise before confirmatory collection.	Preserves interpretability and the planned decision rule.

15. Immediate Next Actions

1. Review PR #17's final diff and merge it when satisfied; verify the successful workflow and update stale PR wording.
2. Choose and pin the TinyStories source revision and GPT-2 BPE tokenizer artifacts. Write a script that downloads, validates and hashes them.
3. Implement the smallest baseline-only training path first; get one deterministic batch and one tiny overfit test working.
4. Add the ASWP module exactly as specified in the protocol and pass zero-noise equivalence before trying nonzero sigma.
5. Run a short matched baseline/ASWP GPU cost pilot. Record real tokens/second, memory and billing.
6. Use measured costs to decide whether a ~10M screening experiment and eight-pair confirmation fit inside the usable credit with a reserve.
7. Do not start confirmatory runs until the protocol checklist is complete and committed.

16. Decision Rules and Non-Goals

This plan is designed to answer a narrow question: **does ASWP improve final validation NLL over ordinary training for this fixed small Transformer and dataset?**

- It does not guarantee a positive result.
- It does not prove universal gains across architectures, scales, tokenizers or datasets.
- It does not justify comparing one ASWP run to an unrelated pretrained checkpoint as causal evidence.
- It does not authorize tuning the confirmatory threshold, seeds, endpoint or metric after seeing results.
- A null or inconclusive result is still useful if the implementation is correct, the study is properly controlled, and all results are reported.

Reference files in the repository

Repository: https://github.com/kragent66-glitch/aswp
PR #17: https://github.com/kragent66-glitch/aswp/pull/17
Statistical Analysis Plan: https://github.com/kragent66-glitch/aswp/blob/main/docs/03-statistical-analysis-plan.md
Preregistration checklist: https://github.com/kragent66-glitch/aswp/blob/main/docs/06-preregistration-checklist.md
Seed manifest: https://github.com/kragent66-glitch/aswp/blob/main/config/manifest.json

Planning document only. Confirm current dataset terms, Modal pricing and resource availability before executing. No GPU run or efficacy result is implied by this document.